

Java ArrayList — Teaching Notes

Expanded explanations + a dry run for every code example, ready to teach from.

Based on notes by Akshit Sharma

1. What is an ArrayList?

An ArrayList is a resizable array. It lives in `java.util`, and it implements the `List` interface, which means it's a resizable-array implementation of a list of elements.

Why does this matter? A normal Java array (`int[] arr = new int[5]`) has a fixed size forever — once you say 5, you can never add a 6th element without creating a brand-new array. An ArrayList behaves like an array on the outside (indexed access, ordered elements) but grows and shrinks automatically underneath as you add or remove items.

One important restriction: an ArrayList can only store objects, not primitives. So you cannot write `ArrayList<int>`. Instead, Java auto-boxes primitives into their wrapper classes — an `int` becomes an `Integer`, a `double` becomes a `Double` — automatically, behind the scenes.

Code

```
import java.util.ArrayList;
import java.util.List;

List<Integer> numbers = new ArrayList<>();
numbers.add(10);
numbers.add(20);
numbers.add(30);
System.out.println(numbers); // [10, 20, 30]
```

Line-by-line explanation:

- `List<Integer> numbers` — we declare the variable using the interface type `List`, not `ArrayList`. This is best practice (explained more in Section 3).
- `new ArrayList<>()` — this actually creates the object. The `<>` is the diamond operator; Java infers it means `ArrayList<Integer>` from the left side.
- `numbers.add(10)` — appends 10 to the end of the (currently empty) list. Internally, 10 is auto-boxed into `Integer.valueOf(10)`.
- `System.out.println(numbers)` — `ArrayList` overrides `toString()`, so printing it shows `[10, 20, 30]`, not some memory address.

Dry Run

Step	Statement executed	State of 'numbers' after
1	<code>new ArrayList<>()</code>	<code>[]</code> (empty list, size = 0)
2	<code>numbers.add(10)</code>	<code>[10]</code> (size = 1)

3	<code>numbers.add(20)</code>	<code>[10, 20]</code> (size = 2)
4	<code>numbers.add(30)</code>	<code>[10, 20, 30]</code> (size = 3)
5	<code>println(numbers)</code>	Output: <code>[10, 20, 30]</code>

2. ArrayList vs Array — Key Differences

This comparison is a classic interview question. Teach it as: 'array is fixed and fast; ArrayList is flexible and convenient.'

Feature	Array	ArrayList
Size	Fixed at creation	Grows / shrinks dynamically
Data type	Primitives & objects	Objects only (auto-boxing)
Syntax	<code>arr.length</code> , <code>arr[i]</code>	<code>list.size()</code> , <code>list.get(i)</code>
Performance	Slightly faster (no overhead)	Small overhead from resizing/boxing
Built-in methods	Very few (via Arrays class)	Many (add, remove, sort, contains...)
Multi-dimensional	Natural (<code>int[][]</code>)	Needs <code>List<List<T>></code>

Teaching tip: ask the class 'when would you prefer a plain array over an ArrayList?' Answer: when the size is known and fixed, and you need maximum performance (e.g. fixed-size numeric buffers, competitive programming with tight time limits).

3. Declaring, Creating & Generics

There are three common ways to create an ArrayList, and understanding generics (the `<T>` in `List<T>`) is essential before going further.

```
// Declare with the interface type (best practice)
List<String> names = new ArrayList<>();

// With an initial capacity hint (still grows if needed)
List<String> names2 = new ArrayList<>(50);

// From an existing collection
List<String> copy = new ArrayList<>(names);

// Diamond operator <> lets the compiler infer the generic type
```

Explanation of each line:

- `List<String> names = new ArrayList<>()` — declaring the variable as `List` (the interface) instead of `ArrayList` (the class) is a best practice called 'coding to an interface'. It means if you later switch to a `LinkedList`, you only change one line.

- `new ArrayList<>(50)` — the 50 is NOT a size limit, it's a capacity hint. It tells Java 'start the internal array at size 50 so you don't need to resize immediately.' The list still starts with `size() == 0`; you can still `add()` beyond 50 and it will resize automatically.
- `new ArrayList<>(names)` — this copy constructor creates a brand-new `ArrayList` containing all elements currently in 'names'. It is a separate list — modifying 'copy' does NOT affect 'names'.

Dry Run

Step	Statement	Effect
1	<code>List<String> names = new ArrayList<>();</code>	<code>names</code> → [], size 0, internal capacity 10 (JVM default)
2	<code>List<String> names2 = new ArrayList<>(50);</code>	<code>names2</code> → [], size 0, internal capacity 50 (no resize needed soon)
3	<code>names.add("Riya"); names.add("Aman");</code>	<code>names</code> → [Riya, Aman]
4	<code>List<String> copy = new ArrayList<>(names);</code>	<code>copy</code> → [Riya, Aman] (independent copy)
5	<code>copy.add("Zoya");</code>	<code>copy</code> → [Riya, Aman, Zoya] but <code>names</code> is still [Riya, Aman]

Why generics? `List<Integer>` guarantees, at compile time, that only `Integer` objects can go into the list. This prevents a runtime `ClassCastException` later and removes the need for manual casting when reading elements back out.

4. Internal Working (Dynamic Resizing)

This is the concept interviewers love to probe. Internally, an `ArrayList` is just a wrapper around a plain `Object[]` array. When you call `add()` and the backing array is full, Java does three things: (1) allocates a new, bigger array — typically about 1.5× the old capacity, (2) copies every existing element into the new array, and (3) lets the old array be garbage collected.

Because resizing happens only occasionally (not on every single `add`), the average cost of `add()` across many insertions works out to $O(1)$. This is called 'amortized $O(1)$ ' — individual resize operations are $O(n)$, but they're rare enough that they don't dominate the average.

Dry Run — growing a list starting at capacity 2, adding 5 elements

Add #	size before	capacity before	Resize triggered?	capacity after
1	0	2	No — room available	2
2	1	2	No — room available	2
3	2	2	Yes → grow to 3	3
4	3	3	Yes → grow to ~4	4

5	4	4	Yes → grow to ~6	6
---	---	---	------------------	---

Walk the class through it out loud: 'list is full (size==capacity) → Java can't fit the new element → allocate bigger array → copy everything over → THEN insert the new element.' The exact growth factor (1.5x) is a JVM implementation detail — for interviews, what matters is knowing WHY add() is amortized O(1), not the exact numbers.

★ IMPORTANT — TEACH THIS: Explain WHY add() is amortized O(1) even though resizing is O(n). This is the #1 follow-up question interviewers ask.

5. Core Methods (add, get, set, remove, contains, size)

Method	Purpose	Example
add(x)	append to end	list.add(10)
add(i, x)	insert at index i	list.add(1, 99)
get(i)	read element at index i	list.get(0)
set(i, x)	overwrite element at index i	list.set(0, 5)
remove(i)	remove by index (int arg)	list.remove(0)
remove(Object)	remove first matching value	list.remove(Integer.valueOf(5))
size()	number of elements	list.size()
contains(x)	true/false membership check	list.contains(5)
indexOf(x)	first index of x, or -1	list.indexOf(5)
isEmpty()	true if size == 0	list.isEmpty()
clear()	remove all elements	list.clear()
sort(cmp)	sort in place	list.sort(null)

The classic pitfall: remove(int) vs remove(Object)

This single line trips up almost every beginner, so spend real time on it in class.

```
List<Integer> list = new ArrayList<>(List.of(10, 20, 30));
list.remove(1); // removes INDEX 1 -> [10, 30]
list.remove(Integer.valueOf(10)); // removes VALUE 10 -> [30]
```

Why the confusion happens: ArrayList has two overloaded remove() methods — remove(int index) and remove(Object o). When you write list.remove(1), Java sees an int literal and picks the index version, so it removes whatever is sitting at position 1 — NOT the value 1. To remove the value 10 (as an object), you must wrap it: Integer.valueOf(10), which forces Java to choose the Object overload.

Dry Run

Step	Statement	list contents	Explanation
1	<code>new ArrayList<>(List.of(10,20,30))</code>	[10, 20, 30]	initial list
2	<code>list.remove(1)</code>	[10, 30]	int argument → removes element AT index 1, which was 20
3	<code>list.remove(Integer.valueOf(10))</code>	[30]	Object argument → searches for and removes the VALUE 10

⚠ Pitfall: If your Integer list holds small values (0–127) and you call `remove(5)` meaning 'remove the value 5', Java will actually delete the element at index 5 instead — this can silently corrupt logic without throwing an error if the list happens to be long enough. Always use `Integer.valueOf(x)` when you mean 'remove this value'.

★ IMPORTANT — TEACH THIS: `remove(int)` vs `remove(Object)` is the single most common ArrayList mistake beginners make. Spend extra time here and make students write both forms themselves.

6. Iterating an ArrayList

There are four common ways to loop through an ArrayList. Which one to use depends on whether you need the index, whether you're modifying the list, and how concise you want the code.

```
// 1) classic for loop (index-based)
for (int i = 0; i < list.size(); i++) {
    System.out.println(list.get(i));
}

// 2) for-each (cleanest for read-only traversal)
for (int v : list) {
    System.out.println(v);
}

// 3) Iterator (needed if you must remove() while looping)
Iterator<Integer> it = list.iterator();
while (it.hasNext()) {
    int v = it.next();
    if (v % 2 == 0) it.remove(); // SAFE removal
}

// 4) forEach with a lambda
list.forEach(v -> System.out.println(v));
```

Explanation of each style:

- Classic for loop — gives you the index `i`, useful when you need to know position, compare neighbors, or loop backwards.

- for-each — most readable when you just need every value and don't care about the index. Cannot safely add/remove elements during this loop.
- Iterator — the ONLY safe way to remove elements while looping. `it.next()` advances and returns the current element; `it.remove()` deletes the element just returned by `next()`.
- `forEach + lambda` — functional style, one line, good for simple print/processing actions.

Dry Run — Iterator removing even numbers from [10, 15, 22, 7, 8]

Call	<code>v = it.next()</code>	<code>v % 2 == 0?</code>	Action	list after this step
1	10	true	<code>it.remove()</code>	[15, 22, 7, 8]
2	15	false	keep	[15, 22, 7, 8]
3	22	true	<code>it.remove()</code>	[15, 7, 8]
4	7	false	keep	[15, 7, 8]
5	8	true	<code>it.remove()</code>	[15, 7]

Final result: [15, 7]. Notice the Iterator keeps an internal cursor and safely adjusts it as elements are removed — that's exactly why it doesn't throw the `ConcurrentModificationException` you'd get from removing inside a for-each loop (see Section 9).

★ IMPORTANT — TEACH THIS: `Iterator.remove()` is the ONLY safe way to remove elements mid-loop. This connects directly to Section 9 — teach them together.

7. Searching in an ArrayList

Two situations: an unsorted list (must check every element, $O(n)$) or a sorted list (can binary search, $O(\log n)$).

```
FUNCTION linearSearch(list, target):
  FOR i FROM 0 TO list.size() - 1:
    IF list.get(i) == target: RETURN i
  RETURN -1

// Or simply: list.indexOf(target) / list.contains(target)
```

This is pseudocode for a linear search — it checks each element in order until it finds the target or reaches the end. In real code, you'd just call `list.indexOf(target)` (returns index or -1) or `list.contains(target)` (returns true/false); both do this same linear scan internally.

Dry Run — `linearSearch([4, 9, 2, 7, 5], target = 7)`

i	<code>list.get(i)</code>	Match target 7?	Action
0	4	No	continue
1	9	No	continue
2	2	No	continue

3	7	Yes	RETURN 3
---	---	-----	----------

Result: the function returns 3 (index found), and the loop never reaches $i = 4$.

For a SORTED ArrayList, `Collections.binarySearch(list, target)` gives $O(\log n)$ lookup — same idea as binary search on a plain array: repeatedly cut the search range in half.

8. Sorting an ArrayList

```
List<Integer> nums = new ArrayList<>(List.of(5, 3, 8, 1, 9));
Collections.sort(nums); // ascending, natural order
nums.sort(Comparator.reverseOrder()); // descending

// Custom objects: sort by a field using a Comparator
List<String> names = new ArrayList<>(List.of("Charlie", "Alice", "Bob"));
names.sort(Comparator.comparing(String::length)); // by length
names.sort((a, b) -> a.compareTo(b)); // alphabetically
```

Explanation:

- `Collections.sort(nums)` — sorts in place using natural ordering (ascending for numbers, alphabetical for Strings). Java's ArrayList uses TimSort internally, a stable, $O(n \log n)$ hybrid of merge sort and insertion sort.
- `Comparator.reverseOrder()` — a ready-made Comparator that reverses natural order, giving descending order.
- `Comparator.comparing(String::length)` — builds a Comparator that compares Strings by their `length()` value instead of alphabetically.
- `(a, b) -> a.compareTo(b)` — a lambda expression implementing Comparator manually; equivalent to natural (alphabetical) String ordering.

Dry Run — `Collections.sort([5, 3, 8, 1, 9])`

Stage	State
Start	[5, 3, 8, 1, 9]
TimSort compares & merges runs internally (stable, $O(n \log n)$)	...intermediate merging happens here...
Final	[1, 3, 5, 8, 9]

Output after `Collections.sort(nums)`: [1, 3, 5, 8, 9]. If you then run `nums.sort(Comparator.reverseOrder())`, it becomes [9, 8, 5, 3, 1].

Dry Run — sorting names by length: ["Charlie", "Alice", "Bob"]

Name	length()
Charlie	7
Alice	5

Bob	3
-----	---

Sorted ascending by length → [Bob, Alice, Charlie]. If instead we sort alphabetically with (a, b) -> a.compareTo(b), the result is [Alice, Bob, Charlie].

9. Removing Elements Safely While Iterating

Modifying a list (adding or removing elements) with a for-each loop, or with a raw for loop that isn't careful about indices, while iterating over it causes a ConcurrentModificationException, or silently skips elements. This happens because for-each uses an internal Iterator that detects the list was changed outside of its own remove() method, and throws an exception to protect you from unpredictable behavior.

```
// WRONG - throws ConcurrentModificationException
for (int v : list) {
    if (v == 3) list.remove(Integer.valueOf(3));
}

// RIGHT - iterate backwards
for (int i = list.size() - 1; i >= 0; i--) {
    if (list.get(i) == 3) list.remove(i);
}

// RIGHT - use removeIf (cleanest, Java 8+)
list.removeIf(v -> v == 3);
```

Dry Run — WRONG version on [1, 3, 5]

Step	for-each internal action	Result
1	hasNext() → true, next() returns 1	1 != 3, no removal
2	hasNext() → true, next() returns 3	3 == 3 → list.remove(Integer.valueOf(3)) called directly on the list
3	hasNext() called again – Iterator detects the list's modCount changed outside its own remove()	throws ConcurrentModificationException

Dry Run — RIGHT version (backwards loop) on [1, 3, 5, 3, 7]

i	list.get(i)	== 3 ?	Action	list after
4	7	No	–	[1, 3, 5, 3, 7]
3	3	Yes	remove(3)	[1, 3, 5, 7]
2	5	No	–	[1, 3, 5, 7]
1	3	Yes	remove(1)	[1, 5, 7]

0	1	No	—	[1, 5, 7]
---	---	----	---	-----------

Why backwards works: removing an element shifts everything after it one position to the left. If you're iterating backwards, everything you've already visited (to the right) doesn't matter anymore, and everything still to come (to the left) hasn't shifted. Going forwards would cause you to skip the element that slides into the removed slot.

★ **IMPORTANT — TEACH THIS:** `ConcurrentModificationException` is a guaranteed exam/interview question. Make sure students can explain WHY it happens, not just how to avoid it.

Dry Run — `removeIf` on [1, 3, 5, 3, 7]

`list.removeIf(v -> v == 3)` internally scans once and removes every element for which the lambda returns true, handling the index-shifting for you.

Element checked	<code>v == 3 ?</code>	Removed?
1	No	No
3	Yes	Yes
5	No	No
3	Yes	Yes
7	No	No

Final result for all three correct approaches: [1, 5, 7].

10. `ArrayList` of Custom Objects

`ArrayList`s aren't limited to numbers and Strings — they very commonly hold objects of a class you define, like `Student` below.

```
class Student {
    String name;
    int marks;
    Student(String name, int marks) { this.name = name; this.marks = marks; }
    public String toString() { return name + ":" + marks; }
}

List<Student> students = new ArrayList<>();
students.add(new Student("Riya", 88));
students.add(new Student("Aman", 72));
students.sort(Comparator.comparingInt(s -> s.marks)); // ascending by marks
System.out.println(students); // [Aman:72, Riya:88]
```

Explanation: `Comparator.comparingInt(s -> s.marks)` builds a comparator that extracts the 'marks' field from each `Student` and compares those integers, so the list ends up sorted by marks ascending rather than by insertion order.

Dry Run

Step	Action	students list
------	--------	---------------

1	students.add(new Student("Riya", 88))	[Riya:88]
2	students.add(new Student("Aman", 72))	[Riya:88, Aman:72]
3	sort by marks ascending → compare 88 vs 72 → 72 < 88 so Aman comes first	[Aman:72, Riya:88]
4	println(students) calls toString() on each element	Output: [Aman:72, Riya:88]

11. Converting Between Array and ArrayList

```
// Array -> ArrayList
Integer[] arr = {1, 2, 3};
List<Integer> list = new ArrayList<>(Arrays.asList(arr));

// ArrayList -> Array
List<Integer> list2 = new ArrayList<>(List.of(1, 2, 3));
Integer[] arr2 = list2.toArray(new Integer[0]);
```

Explanation: `Arrays.asList(arr)` turns an array into a List *view* backed by that same array — but this view is fixed-size (see the pitfall below), so we wrap it in `new ArrayList<>(...)` to get a fully resizable copy. Going the other way, `toArray(new Integer[0])` copies the ArrayList's contents into a brand-new Integer array of the correct size.

Dry Run

Step	Statement	Result
1	<code>Integer[] arr = {1, 2, 3};</code>	<code>arr</code> → [1, 2, 3] (plain array)
2	<code>Arrays.asList(arr)</code>	a fixed-size List view of <code>arr</code> → [1, 2, 3]
3	<code>new ArrayList<>(that view)</code>	<code>list</code> → [1, 2, 3] (now a real, resizable ArrayList – a copy)
4	<code>list2 = new ArrayList<>(List.of(1,2,3))</code>	<code>list2</code> → [1, 2, 3]
5	<code>arr2 = list2.toArray(new Integer[0])</code>	<code>arr2</code> → [1, 2, 3] (new Integer[] array, independent of <code>list2</code>)

⚠ Pitfall: `Arrays.asList(arr)` returns a fixed-size list backed directly by the array. Calling `add()` or `remove()` on it directly throws `UnsupportedOperationException`, because you can't change the length of the underlying array. Always wrap it in `new ArrayList<>(...)` as shown above if you need a resizable copy.

★ IMPORTANT — TEACH THIS: `Arrays.asList()` being fixed-size surprises almost everyone the first time. Demo the `UnsupportedOperationException` live if possible.

12. 2D ArrayList (List of Lists)

Java doesn't have a built-in 2D ArrayList type the way it has `int[][]` for arrays. Instead, you build a List of Lists — an outer ArrayList where each element is itself an ArrayList.

```
List<List<Integer>> grid = new ArrayList<>();
for (int i = 0; i < 3; i++) {
    grid.add(new ArrayList<>());
    for (int j = 0; j < 3; j++) {
        grid.get(i).add(i * 3 + j);
    }
}
System.out.println(grid); // [[0,1,2],[3,4,5],[6,7,8]]
```

Dry Run

i	Action before inner loop	j	Value added ($i*3+j$)	grid state
0	grid.add(new ArrayList<>())	0,1,2	0, 1, 2	[[0,1,2]]
1	grid.add(new ArrayList<>())	0,1,2	3, 4, 5	[[0,1,2],[3,4,5]]
2	grid.add(new ArrayList<>())	0,1,2	6, 7, 8	[[0,1,2],[3,4,5],[6,7,8]]

Final printed result: `[[0,1,2],[3,4,5],[6,7,8]]` — a 3×3 grid, exactly like a 2D array but built from nested ArrayLists.

13. Time Complexity Cheat-Sheet

Operation	Time
<code>get(i)</code> / <code>set(i, x)</code>	$O(1)$
<code>add(x)</code> at end	Amortized $O(1)$
<code>add(i, x)</code> at index / <code>remove(i)</code>	$O(n)$ — shifts elements
<code>contains(x)</code> / <code>indexOf(x)</code>	$O(n)$
<code>Collections.binarySearch</code> (sorted list)	$O(\log n)$
<code>Collections.sort</code> / <code>list.sort</code>	$O(n \log n)$

Teaching tip: `get/set` are $O(1)$ because ArrayList is array-backed — direct index access. `add(i,x)` and `remove(i)` are $O(n)$ because every element after index i has to physically shift over by one slot.

★ **IMPORTANT — TEACH THIS:** This entire cheat-sheet is exam gold — make students memorize it and be able to justify each Big-O, not just recite it.

14. Common Mistakes & Interview Tips

- `IndexOutOfBoundsException` — a valid index must satisfy $0 \leq \text{index} < \text{size}()$. Using `index == size()` (or higher) throws this exception; a common off-by-one mistake.
- Confusing `remove(int index)` with `remove(Object o)` for `Integer` lists — see Section 5 for the full dry run.
- Modifying a list while iterating with `for-each` → use `Iterator.remove()` or `removeIf()` instead (Section 9).
- Using `Arrays.asList()` and then trying to add/remove → wrap it in `new ArrayList<>()` first (Section 11).
- Forgetting `ArrayList` can't hold primitives directly — `int` auto-boxes to `Integer`, which carries a small memory/performance cost versus a raw `int[]`.
- Choosing `ArrayList` when you mostly insert/delete at the front → a `LinkedList` (or `ArrayDeque`) may fit better, since those operations are $O(1)$ there versus $O(n)$ for `ArrayList`.
- Always state the Big-O for the specific `ArrayList` operation you used when explaining a solution out loud in an interview — it shows you understand the underlying cost, not just the syntax.

End of expanded notes — every code block above now has a full explanation and a step-by-step dry run to teach from directly.